A) Implementing the 16 CNN Configurations

In this section, we are required to implement the following 16 configurations using CNN:

- Number of CNN layers: 1 or 2
- CNN layer size (number of filters): 16 or 32
- Kernel size (window size): 5 or 7
- MaxPooling: With or without a MaxPooling layer

For each configuration, the model is trained for 5 epochs. The Confusion Matrix and evaluation metrics are then calculated, and the model summary is also included in the output.

The table below presents all 16 models, along with their corresponding Training Accuracy and Validation Accuracy.

Validation Accuracy	Training Accuracy	Model
0.8054	0.9201	Conv Layer = 1 , Filter Size = 16, Kernel Size = 5, No MaxPooling
0.8040	0.9191	Conv Layer = 1 , Filter Size = 16, Kernel Size = 5,With MaxPooling
0.7934	0.9272	Conv Layer = 1 , Filter Size = 16, Kernel Size = 7, No MaxPooling
0.8005	0.9185	Conv Layer = 1 , Filter Size = 16, Kernel Size = 7, With MaxPooling
0.8042	0.9168	Conv Layer = 1 , Filter Size = 32, Kernel Size = 5, No MaxPooling
0.7983	0.9147	Conv Layer = 1 , Filter Size = 32, Kernel Size = 5, With

		MaxPooling
0.8008	0.9321	Conv Layer = 1 , Filter Size = 32, Kernel Size = 7, No MaxPooling
0.8033	0.9179	Conv Layer = 1 , Filter Size = 32, Kernel Size = 7, With MaxPooling
0.8007	0.9312	Conv Layer = 2 , Filter Size = 16, Kernel Size = 5, No MaxPooling
0.7760	0.9238	Conv Layer = 2 , Filter Size = 16, Kernel Size = 5, With MaxPooling
0.7886	0.9441	Conv Layer = 2 , Filter Size = 16, Kernel Size = 7, No MaxPooling
0.7957	0.9275	Conv Layer = 2 , Filter Size = 16, Kernel Size = 7, With MaxPooling
0.8014	0.9389	Conv Layer = 2 , Filter Size = 32, Kernel Size = 5, No MaxPooling
0.7808	0.9196	Conv Layer = 2 , Filter Size = 32, Kernel Size = 5, With MaxPooling
0.8007	0.9408	Conv Layer = 2 , Filter Size = 32, Kernel Size = 7, No MaxPooling
0.7972	0.9372	Conv Layer = 2 , Filter Size = 32, Kernel Size = 7, With MaxPooling

Based on the results, we can conclude that increasing the number of CNN layers improves Training Accuracy, indicating that the model has a greater capacity to learn more complex patterns. However, this does not necessarily lead to higher Validation Accuracy or better generalization.

The kernel size, filter size, and the presence or absence of a MaxPooling layer have a relatively smaller impact on Validation Accuracy compared to the number of CNN layers.

Ultimately, the best-performing configuration is:

- Number of CNN layers: 1
- Filter size: 16
- Kernel size: 5
- MaxPooling: Not used

This configuration achieves higher accuracy than the other models. Therefore, we can conclude that the simplest model provides the best Validation Accuracy, while more complex models appear to suffer from overfitting and achieve lower validation performance.

Furthermore, when comparing these 16 configurations with the model from the previous assignment that used a SimpleRNN layer, we observe that the SimpleRNN model achieves higher Training Accuracy than all 16 CNN configurations. This suggests that adding a SimpleRNN layer resulted in better performance on this particular dataset.

We follow the same procedure for the other two datasets, implementing and executing the corresponding code.

Results for the second dataset:

Overall, we observe that simpler models with one CNN layer perform better than models with two CNN layers.

Among the models with one CNN layer, a kernel size of 7 achieves better performance than a kernel size of 5. The filter size does not have a significant impact when comparing the models. Similarly, adding or removing the MaxPooling layer sometimes improves accuracy and sometimes reduces it.

Overall, the best-performing configuration among the 16 models consists of:

- Number of CNN layers: 1
- Kernel size: 7
- Filter size: 16
- MaxPooling: Used

For this dataset, the highest accuracy achieved is 0.85, which is better than the accuracy of 0.83 obtained in the previous assignment. Therefore, we can conclude that using a CNN layer performs better than using a SimpleRNN layer for this dataset.

Results for the third dataset:

Here, we also observe that models with one CNN layer generally perform better than those with two CNN layers.

Among the models with one CNN layer, a kernel size of 7 generally performs slightly better than a kernel size of 5. Regarding filter size, configurations with 32 filters generally achieve better performance than those with 16 filters.

Furthermore, using MaxPooling does not improve accuracy in this case. Therefore, we can conclude that adding a MaxPooling layer does not necessarily improve model accuracy for this particular task.

The best-performing configuration for this dataset consists of:

- Number of CNN layers: 1
- Kernel size: 7
- Filter size: 32
- MaxPooling: Not used

This configuration achieves an accuracy of 0.995.

By comparing the highest accuracy among these 16 configurations with the accuracy obtained in the previous assignment, we observe that this model performs better than the previous model, which achieved an accuracy of 0.96. Therefore, for this dataset as well, using a CNN layer performs better than using a SimpleRNN layer.

The code for this section is available in File A.

B) Combining CNN with LSTM and Pretrained GloVe Embeddings

In this section, we are required to add the best-performing model from Part A (i.e., one CNN layer) to the model from the previous assignment, which uses GloVe as a pretrained Word Embedding model and includes two LSTM layers. We then evaluate the resulting model.

Results for the IMDB dataset:

After adding one CNN layer, we observe that the model's accuracy is lower than the accuracy achieved in Part A (where the model did not use GloVe or LSTM and achieved an accuracy of 0.85). However, it performs better than the model from the previous assignment, with an improvement of approximately 0.2 in accuracy.

This indicates that adding a CNN layer before the two LSTM layers has improved the model's accuracy to some extent. One possible explanation is that the CNN layer extracts and filters important features from the text and passes them to the LSTM layers for further processing, which can improve the overall accuracy.

Results for the second dataset:

After adding the CNN layer, we observe no significant change in the model's accuracy.

Results for the third dataset:

By adding a CNN layer to the model that uses GloVe embeddings and two LSTM layers, we observe an accuracy improvement of approximately 0.3 compared to the previous assignment.

Overall, we can conclude that adding a CNN layer can improve model accuracy. However, this is not always guaranteed. The results depend on several factors, including the nature of the problem, the specific task, and the dataset. With appropriate hyperparameter tuning and a configuration suited to the problem, adding a CNN layer may improve performance, but, as mentioned earlier, this is not necessarily the case for every dataset.

The code for this section is available in File B.